

## **Teknisk sammendrag**

Dette dokumentet beskriver den tekniske arkitekturen og implementasjonen av PhishShield. Fokus er lagt på hvordan applikasjonen er bygget, hvordan kode og data er strukturert, og hvilke tekniske valg som er gjort underveis i utviklingen.

Løsningen er implementert som en klientbasert mobilapplikasjon uten backend-tjenester og uten nettverkskommunikasjon i runtime. All funksjonalitet kjøres lokalt på enheten, og innholdet er basert på statiske datafiler. Dokumentet er ment som et teknisk supplement til prosjektets sikkerhets- og refleksjonsdokument, og beskriver hvordan tekniske valg støtter prosjektets overordnede mål om enkelhet, kontroll og forutsigbarhet.

## **1. Teknologivalg og rammeverk**

PhishShield er utviklet ved hjelp av React Native og Expo. Valget av disse teknologiene ble gjort for å kunne bygge en plattformuavhengig applikasjon med én felles kodebase, samtidig som utviklingsmiljøet holdes oversiktlig og effektivt.

Applikasjonen er skrevet i JavaScript, med støtte for TypeScript der det har vært hensiktsmessig. For innhold og konfigurasjon er det benyttet JSON-filer. Dette gir en tydelig separasjon mellom applikasjonslogikk og data, og gjør løsningen enkel å vedlikeholde og utvide.

Jeg har bevisst valgt å holde teknologistacken relativt enkel. Målet har ikke vært å demonstrere flest mulig rammeverk eller biblioteker, men å bygge en løsning som er lett å forstå, teste og videreutvikle.

## 2. Overordnet systemarkitektur

Applikasjonen er bygget som et rent klientbasert system. Det finnes ingen backend-tjenester, API-er eller databaser tilknyttet løsningen. All logikk, presentasjon og databehandling skjer lokalt på brukerens enhet.

Denne arkitekturen gir en forutsigbar og deterministisk applikasjon, der funksjonalitet og oppførsel ikke er avhengig av eksterne systemer. Det gjør også løsningen enklere å analysere teknisk, ettersom det ikke finnes skjulte avhengigheter eller nettverksbaserte komponenter i runtime.

Arkitekturen støtter prosjektets formål ved å holde kompleksiteten lav og ved å sikre at applikasjonen fungerer likt uavhengig av nettverkstilgang.

## 3. Kode og mappestruktur

Kodebasen er strukturert med mål om tydelig ansvarsfordeling og god lesbarhet.

Mappestrukturen er valgt for å skille mellom presentasjon, gjenbrukbare komponenter, systemlogikk og data.

| Mappe                | Rolle                                           |
|----------------------|-------------------------------------------------|
| screens/             | Skjermkomponenter og navigasjonsflyt            |
| components/          | Gjenbrukbare UI-komponenter                     |
| hooks/               | Temastyring og systemrelatert logikk            |
| questions.no/en.json | Språkspesifikke datafiler for innhold           |
| scripts/             | Utviklingsverktøy for generering og vedlikehold |

Skjermene er ansvarlige for brukerinteraksjon og visning av innhold, men inneholder minimalt med forretningslogikk. Gjenbrukbare komponenter er isolert for å sikre konsistent

utseende og oppførsel på tvers av applikasjonen. Systemrelaterte funksjoner, som temahåndtering, er samlet i egne hooks for å unngå spredning av global logikk.

Denne strukturen har gjort det enklere å videreutvikle applikasjonen, blant annet ved overgangen til flerspråklig støtte, uten omfattende endringer i eksisterende kode.

## **4. Navigasjon og skjermflyt**

Navigasjonen i PhishShield er bevisst holdt enkel og oversiktlig. Applikasjonen er delt inn i tydelige skjermer med klart avgrensede ansvarsområder, slik at brukeren lett forstår hvor i applikasjonen de befinner seg og hva neste steg er.

Hovedflyten består av:

- En introduksjons- og startskjerm
- Tilgang til læringsinnhold
- Quiz-funksjonalitet
- En enkel sjekklister for praktisk bruk

Navigasjonsstrukturen er lineær og uten skjulte stier. Jeg valgte å unngå komplekse navigasjonsmønstre og dype hierarkier, da dette kunne gjort applikasjonen mindre tilgjengelig for målgruppen. Målet har vært å redusere kognitiv belastning og sørge for at brukeren kan fokusere på innholdet fremfor på hvordan applikasjonen fungerer.

Skjermene er utformet slik at hver enkelt skjerm har ett hovedformål. Dette gjør koden enklere å vedlikeholde, samtidig som brukeropplevelsen fremstår mer forutsigbar.

## **5. Datamodell og innholdsstruktur**

Alt innhold i PhishShield er basert på statiske JSON-filer. Spørsmål, svaralternativer og forklaringer er lagret i språkspesifikke filer ([questions.no.json](#) og [questions.en.json](#)). Dette gir en tydelig separasjon mellom applikasjonslogikk og innhold.

Hver språkfil følger samme struktur, med identiske identifikatorer for spørsmålene. Kun selve tekstinnholdet varierer mellom språkene. Denne løsningen gjør det mulig å bytte språk uten å påvirke logikken i applikasjonen.

**Jeg valgte denne datamodellen fordi den:**

- Er enkel å lese og forstå
- Er lett å validere manuelt
- Ikke krever ekstern datalagring
- Gir full kontroll over innholdet

Ved å bruke statiske datafiler unngås også uforutsigbarhet knyttet til dynamisk innholdsgenerering i runtime. Dette gir en deterministisk applikasjon der samme input alltid gir samme resultat, noe jeg vurderte som viktig for et opplæringsverktøy.

## **6. Internasjonalisering (teknisk)**

Applikasjonen ble opprinnelig utviklet med støtte for kun norsk språk. Dette gjorde det mulig å fokusere på funksjonalitet, struktur og pedagogisk oppsett i en tidlig fase. Etter hvert som applikasjonen tok form, ble det tydelig at løsningen kunne utvides til å støtte flere språk uten større endringer i arkitekturen.

Utvidelsen til engelsk støtte ble gjennomført ved å:

- Flytte alt tekstbasert innhold ut av komponentene
- Samle språkavhengig innhold i egne JSON-filer
- Benytte felles identifikatorer på tvers av språk

Applikasjonslogikken forble uendret under denne prosessen. Dette bekreftet at separasjonen mellom innhold og logikk fungerte som tiltenkt, og bidro til en ryddigere og mer fleksibel kodebase.

Jeg valgte å ikke benytte eksterne i18n-tjenester eller automatiske oversettelsesløsninger. All språkstøtte er implementert lokalt og statisk, noe som gir full kontroll og eliminerer avhengighet til tredjepart i runtime.

## 6.1 Utfordringer

Under arbeidet med flerspråklig støtte ble det avdekket svakheter i den opprinnelige innholdsstrukturen. De norske og engelske JSON-filene hadde ulike identifikatorer, ulik struktur og manglende kategorier. Dette førte til at enkelte skjermer ikke viste innhold, og at brukerinteraksjoner ikke ga forventet respons.

Løsningen ble å standardisere datamodellen fullstendig. Begge språkfiler følger nå identisk struktur, med like kategorinavn og identiske id-er for alle spørsmål. Kun tekstinnholdet varierer mellom språkene. Denne endringen gjorde det mulig å bytte språk uten å påvirke applikasjonslogikken, og bekreftet viktigheten av konsistent datamodell ved bruk av statiske datafiler.

## 7. Temahåndtering og UI-konsistens

Brukergrensesnittet i PhishShield er bygget med fokus på konsistens og forutsigbarhet. For å oppnå dette er farger, typografi og layout håndtert sentralt gjennom egne tema- og hjelpefunksjoner. Dette gjør det mulig å endre utseende og tilpasse visuelle valg uten å måtte gjøre inngrep i hver enkelt skjerm.

Applikasjonen støtter både lys og mørk visning. Valget av visning følger enhetens innstillinger, og alle fargevalg er abstrahert fra komponentene. Jeg har bevisst unngått hardkoding av farger og stilverdier i skjermkomponenter, da dette ofte fører til inkonsistens og gjør videre vedlikehold mer krevende.

Ved å samle temalogikk i egne hooks og gjenbrukbare komponenter har jeg oppnådd et tydelig skille mellom presentasjon og funksjonalitet. Dette bidrar til en mer oversiktlig kodebase og reduserer risikoen for utilsiktede visuelle avvik.

## 8. Generativ kunstig intelligens (teknisk)

Generativ kunstig intelligens er brukt som et støtteverktøy i utviklingsfasen av PhishShield, hovedsakelig til å generere forslag til phishing-scenarioer og til å strukturere innhold. AI-bruken er begrenset til utviklingsmiljøet og er ikke en del av applikasjonens runtime.

Teknisk er dette løst ved at generert innhold behandles som statiske datafiler. Når innholdet først er generert og kvalitetssikret, lagres det i JSON-format og brukes på samme måte som manuelt skrevet innhold. Dette gjør applikasjonen deterministisk og forutsigbar, uten behov for dynamisk generering eller nettverkstilgang.

Jeg valgte denne tilnærmingen for å beholde full kontroll over både innhold og oppførsel. Ved å holde AI utenfor runtime unngås uforutsigbarhet, avhengighet til tredjepart og potensielle sikkerhetsutfordringer knyttet til ekstern databehandling.

## **9. Testing og tekniske begrensninger**

Testing av applikasjonen er gjennomført manuelt under utvikling, med fokus på funksjonalitet, navigasjon og korrekt visning på ulike enheter og skjermstørrelser. Spesiell vekt har blitt lagt på å verifisere at applikasjonen fungerer likt med og uten nettverkstilkobling.

Jeg har ikke implementert automatiserte tester i dette prosjektet. Dette er et bevisst valg basert på prosjektets omfang og karakter. Fokus har vært på å validere brukerflyt og funksjonalitet i praksis fremfor å etablere et omfattende testregime.

Tekniske begrensninger i løsningen inkluderer:

- Ingen backend eller synkronisering mellom enheter
- Ingen lagring av progresjon eller brukerdata
- Ingen dynamisk innholdsoppdatering

Disse begrensningene er vurdert som akseptable, da prosjektets formål er opplæring og demonstrasjon, ikke produksjonsbruk i stor skala.

## **10. Avslutning**

Dette tekniske dokumentet har beskrevet hvordan PhishShield er bygget, hvilke tekniske valg som er gjort, og hvordan disse valgene støtter prosjektets overordnede mål. Løsningen er utviklet med fokus på enkelhet, kontroll og tydelig struktur, fremfor kompleksitet og funksjonsrikdom.

Prosjektet viser min tilnærming til teknisk utvikling: å bygge forståelige systemer med bevisste avgrensninger, tydelig ansvarsdeling og forutsigbar oppførsel. Sammen med sikkerhets- og refleksjonsdokumentet gir dette et helhetlig bilde av både teknisk gjennomføring og sikkerhetsmessig vurdering.